Security Audit Report: PCASH

Auditician
GPT-6 Luna, maximum effort

October 25, 2026

Contents

1 Introduction **1**

2 Threat Model **1**

3 Security & Privacy Findings **2**

3.1 Confirmed issues 2

3.2 Proof-group disposition 2

4 Recommendations **3**

5 Future Work **3**

6 Conclusion **3**

A Audit Method and Evidence **4**

A.1 Coverage and tooling 4

A.2 Test evidence 4

A.3 Operational observation disposition 5

1 Introduction

This report presents a soundness-focused security audit of PCASH, a private-value protocol whose nodes derive state from Ethereum data and verify recursively composed proofs before applying transaction effects. The audited target is PCASH, repository name `pcash`, at target SHA:

`cfcb13e3bfc80d53a5565ed43b496b589695168c`

The corresponding public repository is `pcash-protocol/pcash` at public commit:

`bc289e7d43d52dcd5d06f5f3a8ecef9b9cbf8ede`

Internal documents omitted from the public tree were also read as context. The protocol specification is the normative protocol input for this engagement; explanatory internal documents were not treated as normative requirements.[3, 1]

The review focused on whether accepted proofs and signatures authorize the effects that nodes apply. It covered protocol and reference-rule circuits, signature and hash code, recursive proof composition, release generation and pins, the verifier, transaction parsing and derivation, issuance, L1 and data-availability handling, and canonical state. The generated scope inventory contains 204 in-scope files, 1,588 excluded files, and zero unmatched tracked files. The SDK, marketplace, token and lineage applications, hosts, most node services, and general availability behavior were deferred; their guarantees are not certified by this report.

The audit found no confirmed in-scope security issues. The review supported or conditionally proved properties SP1–SP10 and SP12 under the assumptions in the relevant proof records. SP11's broader claim about every ambiguous storage error did not pass its attempted proof, but the resulting issue was classified invalid for this audit: the reproduced stale-view behavior required a local storage fault, the node halted, restart recovered the durable block, and no V7/N1 violation or attacker path was demonstrated.

The five proof groups cover value and settlement (SP1–SP3), account authority (SP4–SP5), signed intent and delivery (SP6–SP7), recursive composition and statement reconstruction (SP8–SP9), and release identity, state recovery, and node agreement (SP10–SP12). These are conditional arguments with executable evidence, not machine-checked theorems or a certification of the proof system.

This audit and this report were AI-generated and may have missed bugs.

2 Threat Model

The finalized ICTM model describes the system broadly and this engagement's narrower scope specifically. It names any Ethereum sender, malicious rule/event author, counterparty, relayer, block builder, peer, client, malicious node, and tagged key compromises as adversaries. The core attacker considered here can control transaction bytes, proofs, witnesses, carrier events, and custom applications, but cannot rewrite the audited source/build machine, break Ethereum consensus or the configured execution RPC assumption, or break the pinned proof-system cryptography. The trust assumptions and stable invariant IDs below follow the threat model.[2]

Money and settlement: V1–V7, L1.
: Accepted actions conserve native value and asset value, fixed-supply assets issue once, nullifiers prevent repeated spends, issuance follows the capped integer schedule, failed items do not apply effects, and recovered state follows the canonical history.

Account and signing authority: A1–A6, A8–A9, R2.
: Note spends use the relevant account/rule authority; account updates use the live row; reference approvals bind their specified effects; and the reference rules constrain change under their stated policy.

Protocol and release binding: D1–D13, D15.
Action, state, event, carrier, transaction type, profile, hash encoding, recursive links, public statements, and release artifacts remain bound across circuit and node boundaries. D14 additionally requires historical-result consumers to authenticate the history root and check result semantics. The helper was reviewed; production consumers and proof-serving paths were deferred, so D14 is not marked passed end to end.

Node agreement and integrity: N1, N10.
Nodes using identical canonical input, configuration, rules, and pinned release derive the same accepted results; verifier and blob integrity checks reject mismatches.

The main trust boundaries are Ethereum consensus and data availability, the configured execution RPC, and the cryptographic assumptions of pinned Barretenberg CHONK. Circuit logic, recursive composition, verifier integration, generated release pins, and the completeness of the release checks remain audit targets rather than assumed-correct components. The public tree does not include all internal documents supplied with the target packet; these were read as context, while the protocol specification remains the normative source.

The model explicitly defers privacy (P1–P9), wallet recovery and wallet-engine properties, general node availability and public surfaces, marketplace, token and lineage guarantees, and Ethereum submission paths. Deferred properties are neither passed nor accepted by this report.

3 Security & Privacy Findings

3.1 Confirmed issues

No confirmed security issues were recorded for the audited target and scope. The confirmed-issues directory is empty. The sole issue file ended in the invalid directory and is discussed as an operational observation under Recommendations; it is not counted as a vulnerability or finding.

3.2 Proof-group disposition

Property group	Disposition	Issues
SP1–SP3	Value conservation, fixed-supply assets, capped issuance, equal settlement, and no effects from rejected actions were supported under the stated reachable-state, arithmetic, cryptographic, and release assumptions.	None
SP4–SP5	Account spending authority and live-row sequential account updates were supported under the account-root, hash-binding, and pinned proof assumptions.	None
SP6–SP7	Reference signed projections, replay controls, event linkage, and carrier matching were supported under the signature, CHONK, and trusted-RPC assumptions. Event semantics remain an application responsibility.	None
SP8–SP9	S/L recursive composition and statement/effect reconstruction were supported by source review and pinned native mutation controls.	None
SP10, SP12	Production release/verifier paths and deterministic derivation were supported for the stated configuration and operator assumptions. Cache/verifier pairing remains a caller invariant.	None
SP11	The attempted broad property did not pass for a fault-injected ambiguous durable commit error. The observation was classified invalid as an in-scope security issue; normal recovery paths recover the durable state, and the node halts rather than making another acceptance decision.	Invalid

These results are conditional and do not certify arbitrary third-party policy, wallet code, proof-system soundness, or deferred consumer paths.

4 Recommendations

- Keep the pinned S and L recursive proving, VK and databus mutation tests in release validation. The two profiles have different statement widths and stack links, and should remain reported separately.
- Keep the explicit runtime known-answer set synchronized with every hiding-kernel key selectable by the compiled rule schedule. The current list matches the four current launch keys, but it is not generated mechanically from the circuit catalog.
- Preserve the verifier/cache pairing and block-scoped cache lifetime in every production derivation path. The cache key covers VK, proof, and public inputs but not verifier identity.
- Preserve the live-row compare-and-swap check, checked revision increment, and same-block account overlay together when changing account updates. The revision-exhaustion branch was reviewed statically; a state fixture at `u32::MAX` was not exercised.
- Keep the user-facing boundaries visible: reference signatures do not authorize event semantics or carrier selection, relay owner commitments are opaque to signers, and event claims require consumer-side policy checks.
- Decide whether a node marked halted after a storage error should continue serving canonical state reads. A fault-injected redb 4.2.0 commit error after the durable write left the live process on its previous read view until shutdown, while reopening recovered the new block. The node exposed `derivation=halted` and performed no further derivation. This is an operational observation, not a confirmed security finding: no remote trigger, V7 violation, or different accepted outcome was demonstrated.

5 Future Work

The most important open boundary is D14's production consumer path. This engagement inspected the in-scope result-opening helper, but deferred proof-serving and consuming applications. A future review should verify that each consumer authenticates the history root under the recent header and checks success, transaction type, statement hash, and the output index it relies on.

Future audits should also cover the deferred SDK and wallet engine, host interfaces, node RPC/P2P and availability surfaces, marketplace, token and lineage applications, and end-to-end integration paths where their guarantees depend on protocol results. The current engagement does not certify those components.

Operational follow-up can decide whether canonical reads should become unavailable once storage has halted after an ambiguous commit error. The reproduced process recovered the durable block after reopen, but the in-process view remained stale while the halted status was exposed. The issue is outside the soundness scope and is not a confirmed vulnerability.

Coverage limits include the following: the non-host platform tool binaries were hash-checked but not executed; the account-revision overflow branch was source-reviewed but not tested with a maximum-revision fixture; some derivation tests use a stub verifier to isolate state transitions; one live-Sepolia test and several verifier/native-proof tests were ignored by default; and the audit is not a formal proof of all possible statements, histories, or storage failures.

6 Conclusion

At target SHA `cfcb13e3bfc80d53a5565ed43b496b589695168c`, the audit found no confirmed security issue in the defined proof-soundness scope. Source review, pinned builds, independent encoding and issuance checks, Rust integration tests, and native S/L proof mutation tests supported the key value, authority, signing, recursive-composition, release, and deterministic-derivation properties under their stated assumptions. The result remains bounded by the audit's trust model and explicit exclusions; deferred components and D14's end-to-end consumer boundary remain open work.

A Audit Method and Evidence

A.1 Coverage and tooling

The local file review completed all 204 in-scope files; 1,588 tracked files were excluded by the engagement scope, and no tracked file was unmatched. Thirty-eight cross-component global reviews and five proof groups examined overlapping seams in value conservation, account authority, signature binding, carrier provenance, recursive links, statement reconstruction, issuance, release identity, state persistence, and deterministic outcomes. No command or build was run in the audited target checkout; tests used isolated scratch checkouts at the target SHA.

The pinned compiler was Nargo 1.0.0-rc.3 (noirc commit `5a7ee9bf5ed8973076df7bb0d2b723024db09ae7`); the pinned native backend was Barretenberg 6.0.0. The full pinned release check compiled protocol and catalogued circuits, generated nine known-answer tests, checked circuit-size and SRS requirements, and matched 82 generated manifest/VK/prover-artifact files against the bundle. The generated release pins check passed. Four platform archive and binary hashes and the installed CRS hashes were checked; non-host binaries were not run. The checks support release identity and reproducibility for this target and environment, not proof-system soundness.

A.2 Test evidence

- **Circuit unit and compile checks.** Pinned Nargo tests passed for the protocol library (194), signature library (9), shared reference-rule library (76), payment (19), admin (11), asset payment (12), and asset burn (16). The S/L asset creation, batch transfer, sponsorship, and several kernel wrappers compiled, but some have zero local test functions; compilation is not counted as a test pass.
- **Native recursive proofs.** With the pinned Barretenberg backend and prover-required test settings, the final recursive-link mutation suite passed 41/41 for S and 26/26 for L; the event-composition suite passed 26/26 across S/L; and direct event-family kernel mutations passed 4/4. These suites included valid controls and child key, statement, action, anchor, event, and databus substitution cases. An initial L sponsored-transfer attempt used an S-shaped sponsor fixture and was inconclusive; only the corrected L fixture run is counted.
- **Rust node and state tests.** The state suite passed 98 tests (70 library, 10 codec, 18 integration); node reorg tests passed 56/56; node pipeline tests passed 12/12; and derivation tests passed 200 with four native-proof tests ignored. L1 tests passed 50 with one live-Sepolia RPC test ignored. Data availability with KZG passed 13 tests, and crypto tests passed 19.
- **Verifier tests.** The default verifier suite passed 41 with 12 ignored. A separate explicit run of the ignored startup known-answer test passed 1/1 using the pinned native verifier and CRS. The other ignored verifier tests are not counted as passes.
- **Issuance and independent encoding checks.** An independent arbitrary-precision Python oracle matched 30 frozen issuance transitions and all 8,519,681 legal Q16 exponents. Independent Poseidon2 checks matched nine field vectors, seven byte-hash boundary vectors, and 46 domain tags; independent Keccak checks covered seven input lengths including the 136-byte rate boundary. An EIP-712 mutation harness compared 12 payloads and 346 message-field plus 48 domain-field mutations against ethers.
- **Release regression suite.** The final suite passed 45/45 with `TMPDIR` set to `/private/tmp/codex` when running `npm run circuit-bundle:test`. An earlier run with the runtime's default temporary directory failed the Cargo re-resolution case; setting the explicit writable temporary directory resolved it. Only the final 45/45 run is counted as passing.

The counts above are distinct suites, not a single aggregate test total. Several integration tests use accepting verifier stubs to isolate ordering and state transitions; separate pinned native tests exercise proof composition and public-input refusal. No skipped or inconclusive test is reported as passing.

A.3 Operational observation disposition

The invalid issue `ambiguous-redb-commit-live-stale.md` documents a reproduced stale view after a local storage backend reports an error after the durable redb commit. The in-memory node rolled back and halted; reopening recovered the committed block. The PoC did not show a remote attacker trigger, a transaction accepted on divergent state, lost nullifiers, a V7 recovery violation, or an N1 disagreement for nodes completing the same transition. Its observed behavior is noted for an operational decision only and is not a security finding.

References

[1] Auditician. PCASH Audit Instructions. Audit packet, 2026. Engagement scope, trust assumptions, and reporting requirements.

[2] Auditician. PCASH Invariant-Centric Threat Model. Audit packet, 2026. Final threat model for this audit run.

[3] Auditician. PCASH Protocol Specification. Audit packet, 2026. Normative protocol input at the audited target SHA.